

BEGINNER GUIDE · SLIDES

Inside CareerHub

The `src` folder

The front-end, room by room — with real screenshots from the running app, and how it connects to a C# back-end.

Next.js 15

React 19

TypeScript

Tailwind CSS

`src` — the whole front-end

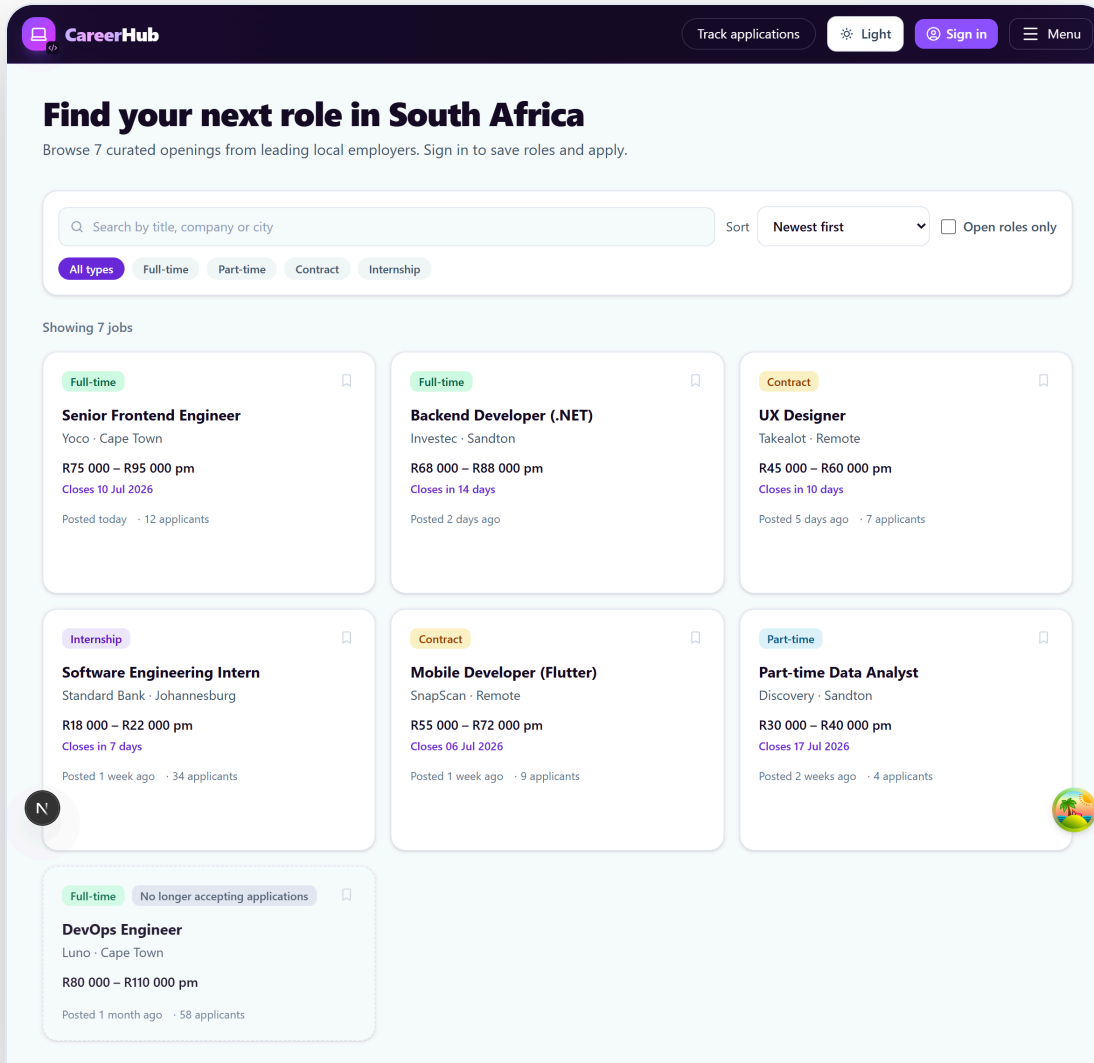
One container, six rooms. Each has a single clear job.

`src/`

- └ `app/` – pages & routes
- └ `components/` – reusable UI blocks
- └ `context/` – shared memory (login, toasts)
- └ `fonts/` – the typefaces
- └ `lib/` – logic, data, formatting
- └ `types/` – the data contract

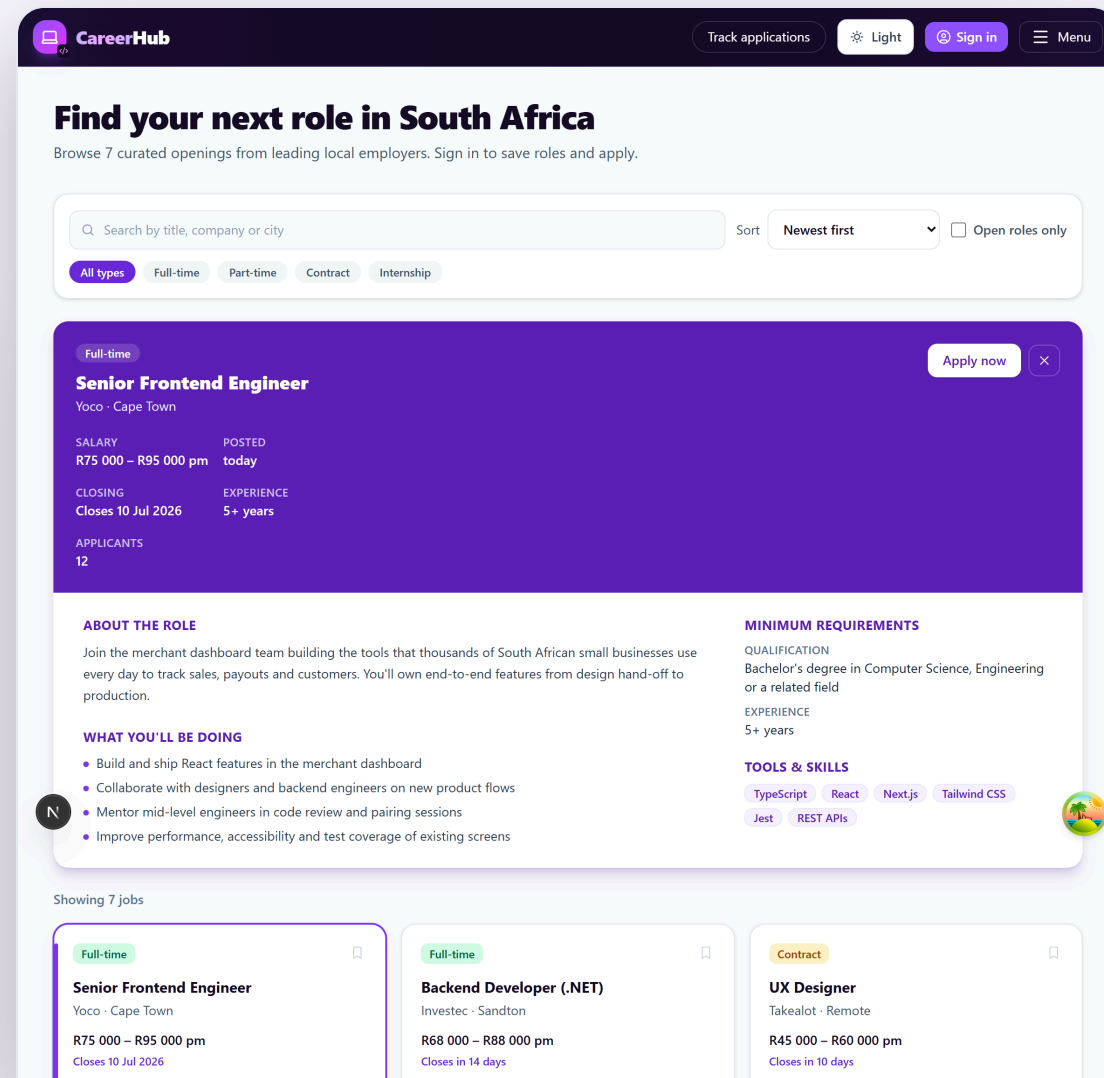
A website is like a house. **src** is the whole house; each folder is a **room** with one responsibility. Good organisation is what lets a team build something large without chaos.

Folders become web pages

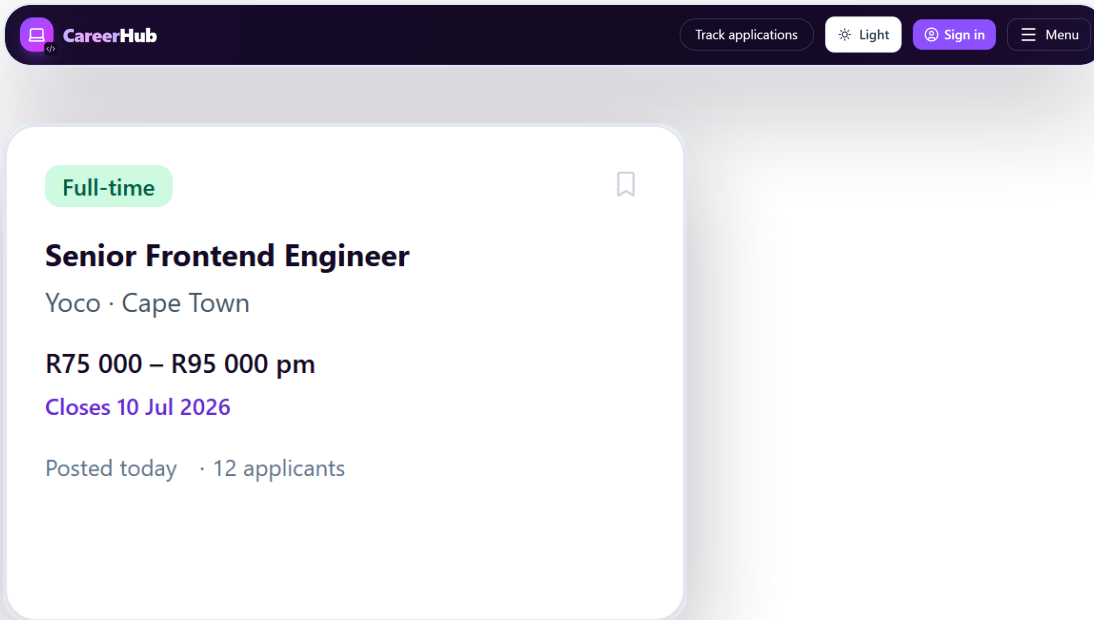


- ▶ **File-based routing:** a folder = a URL
- ▶ `page.tsx` is the screen at that address
- ▶ `layout.tsx` is the frame around every page (navbar + footer)
- ▶ The page shows **loading** → **error** → **success** states

Same page, live states



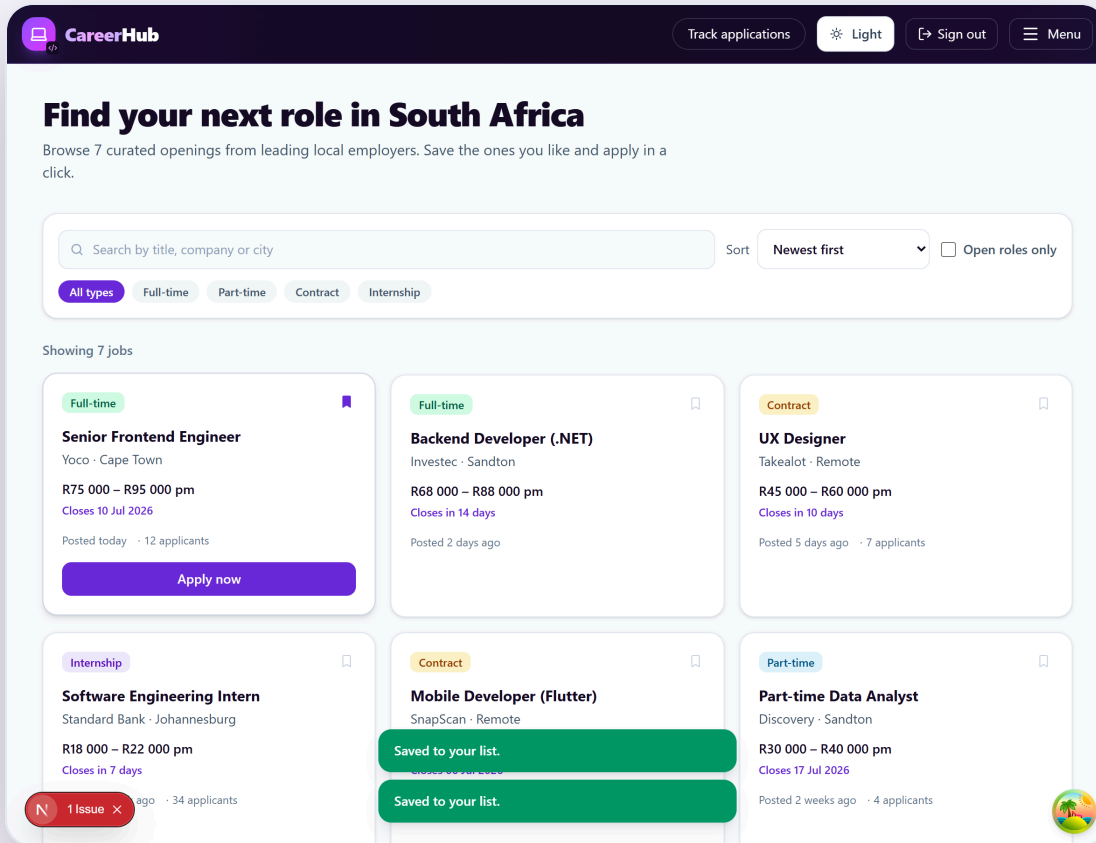
Reusable building blocks



REAL — `Navbar.tsx` & `JobCard.tsx`

- ▶ A **component** is a LEGO brick — write once, reuse everywhere
- ▶ **Props** are its inputs (which job, is it selected?)
- ▶ It **delegates**: salary & dates from `lib/format.ts`
- ▶ Fix the brick once → every copy updates

Shared memory for the whole app



- ▶ `useAuth()` — who is logged in, everywhere
- ▶ `useToast()` — pop-up messages from anywhere
- ▶ No "prop drilling" through every layer
- ▶ **Swap-ready:** mock login today, real C# server later — UI unchanged

REAL — a live "Saved to your list." toast; navbar shows "Sign out"

The typefaces

Find your next role

↑ Plus Jakarta Sans — headings & logo

Browse curated job listings across South Africa. Save the ones you like and apply in a click.

↑ Inter — body text

Self-hosted `.woff2` files → **private, fast, offline-safe**. Loaded by `app/layout.tsx`.

The engine room

`api.ts`

fetches jobs over the network (door to the back-end)

`format.ts`

`45000` → "**R45 000 pm**"; dates → "2 days ago"

`careerhub-store.ts`

saves applications in the browser (swap-ready)

`employmentType.ts`

one place mapping `"FullTime"` → "Full-time"

`utils.ts`

`cn()` — safely merges styling classes

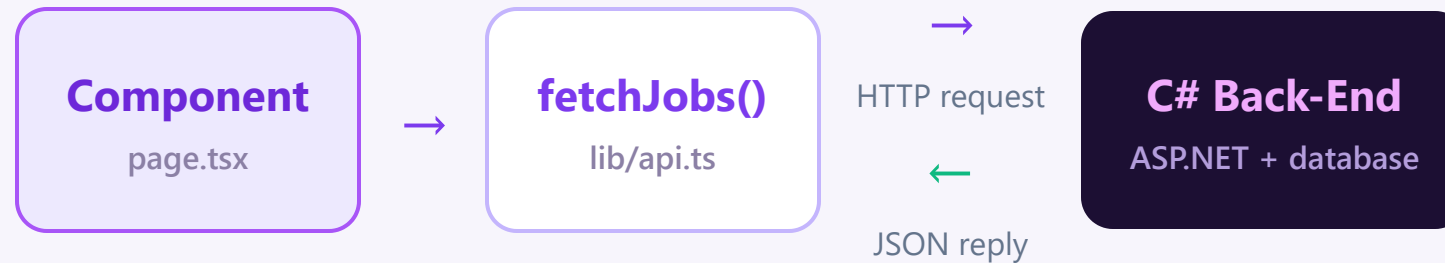
`lib` = how things **work**. `components` = how things **look**.

The data contract



If the back-end renames a field, the front-end **stops compiling** until it's fixed — a bug caught on the developer's screen, never on the user's.

Front-End → C# Back-End



- ▶ They never share code — they swap **HTTP + JSON** messages
- ▶ JSON is text both C# and TypeScript understand → the language doesn't matter
- ▶ Going live = change **one address**: `NEXT_PUBLIC_API_URL`

The front-end folders

- ▶ **Separation of concerns** — looks, logic, data shape & shared state each kept apart
- ▶ **Reuse** — write a card once, draw it many times
- ▶ **Swap-readiness** — demo data → real server with tiny changes

Click a card → page reads it (app) → drawn by bricks (components) → formatted by `lib` → shape guaranteed by `types` → login from `context` → in the right `fonts`. Every room plays its part.

Connecting to the C# back-end

- ▶ **HTTP** carries the request & response
- ▶ **JSON** is the shared language both sides read
- ▶ The **contract** (`types` ↔ `JobListingResponse.cs`) keeps them in step
- ▶ One seam — `fetchJobs()` — is the only door to the server
- ▶ Back-end handles **CORS** + **auth tokens** for a secure, live connection

The UI was finished with mock data before the back-end existed — then flipped to live with a one-line change. That's the pay-off of clean separation.

Thank you.

CareerHub · the `src` folder, explained for beginners.
Full written guide + 40 interview Q&A in the companion PDF.